

KGP Gyankosh: Enterprise Knowledge Assistant

Complete Codebase File Directory, Usage Specifications & Diagrammatic Architecture Workflow

Institution: Kashmir Government Polytechnic College, Srinagar (Directorate of Skill Development, J&K;)

Project: IIT Patna — Generative AI & Agentic AI Capstone (Advanced RAG)

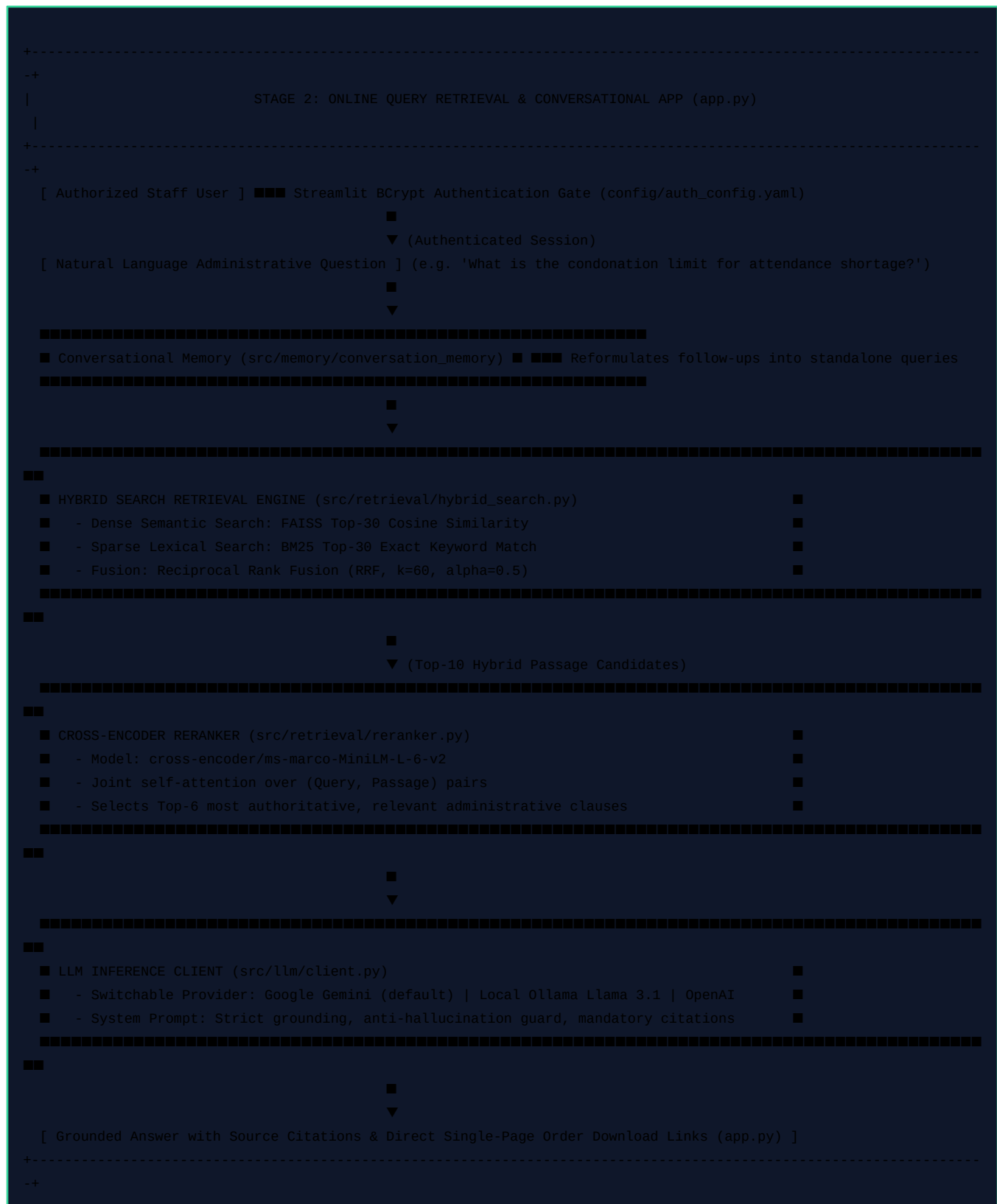
Scope: Full technical reference of all active scripts, config files, pipelines, and data stores.

1. Executive Overview & Two-Stage System Architecture

KGP Gyankosh is an internal administrative intelligence and document query platform built specifically for Kashmir Government Polytechnic (KGP) College, Srinagar. The system operates on a strict **Two-Stage Architecture**, completely decoupling the offline batch indexing pipeline from the online real-time conversational retrieval application. This guarantees that heavy operations (OCR extraction, document loading, chunking, and dense vector generation) occur offline once, ensuring sub-second retrieval latency and zero indexing overhead during staff queries.

2. Diagrammatic Workflows: Two-Stage System Architecture

The diagrams below depict the complete lifecycle of administrative documents from offline batch ingestion to online grounded conversational retrieval.



3. Master File Inventory & Usage Directory

The table below categorizes every file in the KGP Gyankosh repository, its exact path, subsystem role, and operational function.

File Path	Subsystem / Category	Operational Function & Purpose
<code>app.py</code>	Web Interface & Serving	Main Streamlit application. Provides modern glassmorphic UI, BCrypt login gate, dual operational modes (RAG vs Direct LLM), chat memory, direct single-page order download resolution, and session state management.
<code>build_index.py</code>	Offline Indexing CLI	Command-line batch pipeline. Discovers all documents in <code>data/</code> , computes MD5 hashes for incremental indexing, runs text extraction and OCR, splits chunks, and serializes FAISS & BM25 indices.
<code>.env</code>	System Configuration	Active environment configuration. Defines active LLM provider (Ollama, Gemini, OpenAI), API keys, data directory (<code>DATA_DIR=data</code>), embedding models, reranker settings, and retrieval thresholds.
<code>.env.example</code>	Config Template	Documented reference template showing all configurable environment variables with explanations for evaluators and developers.
<code>.streamlit/config.toml</code>	Streamlit Server Config	Configures Streamlit server settings: enables static file serving (<code>enableStaticServing=true</code>), headless mode, disabled file watchers, and primary dark institutional color tokens.
<code>config/auth_config.yaml</code>	Authentication & Security	Active role-based access control (RBAC) credential store. Contains user profiles, salted BCrypt password hashes, and access roles (admin, staff, faculty). Git-ignored for security.
<code>config/auth_config.yaml.example</code>	Auth Template	Example credential configuration with pre-computed BCrypt hashes for evaluation and demonstration accounts.
<code>requirements.txt</code>	Dependency Management	Complete list of pinned Python dependencies: streamlit, langchain, faiss-cpu, rank-bm25, sentence-transformers, google-genai, rapidocr-onnxruntime, pypdf, python-docx, etc.
<code>scratch_test_queries.py</code>	Validation & Test Harness	Standalone testing script for validating FAISS vector retrieval, BM25 keyword matching, and cross-encoder reranking accuracy against sample administrative questions.
<code>src/ingestion/document_loader.py</code>	Ingestion Pipeline	Polymorphic document loader. Automatically detects file types (<code>.docx</code> , <code>.pdf</code> , <code>.txt</code> , <code>.png/.jpg</code>) and dispatches to appropriate parsers. Routes scanned PDF pages directly to OCR.
<code>src/ingestion/ocr_loader.py</code>	OCR & Page Extraction	Optical Character Recognition engine using Pure-Python RapidOCR (ONNX Runtime) with optional Tesseract fallback. Manages persistent page OCR caching in <code>output/ocr_cache/</code> .
<code>src/ingestion/chunker.py</code>	Text Chunking	Recursive character text splitter. Splits documents into 500-character segments with 100-character overlap while strictly respecting paragraph, sentence, and administrative order boundaries.
<code>src/indexing/embeddings.py</code>	Embedding Abstraction	Unified embedding interface supporting local HuggingFace SentenceTransformers (all-MiniLM-L6-v2) and cloud OpenAI embeddings (text-embedding-3-small).
<code>src/indexing/vector_store.py</code>	Vector Store Engine	FAISS vector store manager. Handles dense vector indexing, cosine similarity persistence, on-disk serialization (<code>index.faiss</code> , <code>index.pkl</code>), and fast sub-second index loading.

src/retrieval/hybrid_search.py	Hybrid Retrieval & RRF	Orchestrates dual retrieval: FAISS dense semantic search and BM25 sparse keyword search. Combines ranked candidate lists using Reciprocal Rank Fusion (RRF, k=60, alpha=0.5).
src/retrieval/reranker.py	Neural Cross-Encoder	Deep self-attention reranker (cross-encoder/ms-marco-MiniLM-L-6-v2). Scores (query, passage) pairs simultaneously to eliminate false positives and promote the most authoritative clauses.
src/memory/conversation_memory.py	Conversational Memory	Tracks multi-turn dialogue context up to 6 turns. Prompts the LLM to rewrite ambiguous follow-up questions ("What is the deadline for it?") into complete standalone search queries.
src/llm/client.py	Switchable LLM Engine	Multi-provider inference client supporting Google Gemini, Ollama (Llama 3.1), and OpenAI. Injects grounding guardrails and strictly enforces document title and page number citations.
src/auth/authenticator.py	Auth & Session Security	Wraps streamlit-authenticator to deliver BCrypt cookie-based session management, secure login forms, and role verification without storing plaintext credentials.
data/word/ (*.docx)	Active Administrative Data	Contains 1,157 individual official college Word circulars, notices, committee orders, fee schedules, and leave sanctions issued by Kashmir Government Polytechnic College.
data/pdf/Orders.pdf	Historical Orders Book	85-page comprehensive official scanned orders compilation covering administrative decisions, transfer sanctions, disciplinary actions, and committee notifications.
output/index_manifest.json	Indexing State Manifest	Persistent JSON catalog storing file paths, MD5 file hashes, chunk counts, and timestamps for all 1,158 indexed files. Enables instantaneous incremental updates.
output/ocr_cache/Orders_pdf_pages.json	Persistent OCR Cache	Cached OCR transcriptions for all 85 pages of Orders.pdf. Ensures that OCR extraction runs once and never repeats during index builds or rebuilds.
output/vector_store/ (faiss,.pkl)	Dense Vector Index	Pre-computed FAISS vector index (8,166 embedding vectors) and pickled document metadata chunk store for rapid similarity retrieval.
output/bm25_store/ (model, corpus)	Lexical BM25 Index	Serialized BM25 token frequencies, inverse document frequency statistics, and corpus pickle for exact administrative code and number matching.
static/extracted_pages/ (*.pdf)	Single-Page PDF Cache	Pre-extracted lightweight single-page PDFs for each order in Orders.pdf (e.g. Orders_page_67.pdf). Served directly to users when they click a cited order in the interface.

4. Key Operational Features & Technical Implementations

4.1. Single-Page PDF Order Extraction & Direct Download

A major usability challenge in educational record retrieval is handling multi-page consolidated order books (such as data/pdf/Orders.pdf, a 36 MB compilation of 85 individual administrative orders). In previous versions, clicking an order downloaded the entire 36 MB file. In the current implementation, the function `get_document_download_info()` intercepts citations from multi-page PDFs. Using `pypdf`, it dynamically extracts (or serves from the pre-cached directory `static/extracted_pages/`) a lightweight (~400 KB) single-page PDF containing **only that specific cited order page** (e.g., `Orders_page_67.pdf`). Word documents (.docx) are standalone files and are downloaded directly in their entirety.

4.2. Hybrid Search (FAISS + BM25) & Reciprocal Rank Fusion

Administrative polytechnic circulars contain both high-level thematic concepts (e.g. 'attendance condonation rules', 'student grievance procedure') and highly specific official tokens (e.g. order numbers like KGP/EST/2026/301, committee members, fee amounts like Rs. 2,200). Pure vector search frequently misses exact numeric order numbers, while pure BM25 fails on semantic paraphrasing. KGP Gyankosh combines dense vector similarity (FAISS with all-MiniLM-L6-v2) and sparse lexical scores (BM25) using **Reciprocal Rank Fusion (RRF)** with a constant k=60 and balanced weighting (alpha=0.5). This ensures that documents ranking high in either semantic meaning OR exact token matches rise to the top.

4.3. Cross-Encoder Passage Reranking

While bi-encoders (FAISS) independently embed query and passages for sub-second retrieval, they do not model token-to-token interactions between the query and candidate documents. The cross-encoder (cross-encoder/ms-marco-MiniLM-L-6-v2) takes the top-10 hybrid candidates and computes deep cross-attention over (Query, Passage) pairs simultaneously. This filters out irrelevant text segments before prompt assembly, ensuring the LLM is supplied only with the 6 most authoritative, factually relevant paragraphs.

4.4. Security, BCrypt Authentication & Intranet Deployment

The system is protected by a session-based BCrypt authentication layer (src/auth/authenticator.py) reading salted password hashes from config/auth_config.yaml. The application is configured for local college intranet (LAN) deployment (e.g., http://192.168.x.x:8501), ensuring sensitive administrative personnel records and government circulars remain strictly within the campus perimeter.

5. Step-by-Step Runtime Execution Flow (Query to Answer)

Step	Component	Action & Transformation	Output Artifact
1	User Interface	User enters query via Streamlit chat bar in app.py.	Raw query string
2	ConversationMemory	Examines previous 6 dialogue turns. If contextual follow-up detected, LLM reformulates into standalone query.	Reformulated query string
3	HybridRetriever	Queries FAISS (dense 384-dim cosine) & BM25 (token BM25Okapi). Combines scores via RRF (k=60).	Top-30 hybrid passages
4	DocumentReranker	Cross-Encoder scores (Query, Passage) pairs via deep self-attention.	Top-6 ranked passages
5	LLMClient	Injects grounded system prompt with passages into active LLM (Gemini / Ollama / OpenAI).	Grounded answer with citations
6	Download Resolver	get_document_download_info() creates clickable download links. For PDFs, extracts single page (Orders_page_X.pdf).	Interactive UI + downloadable file